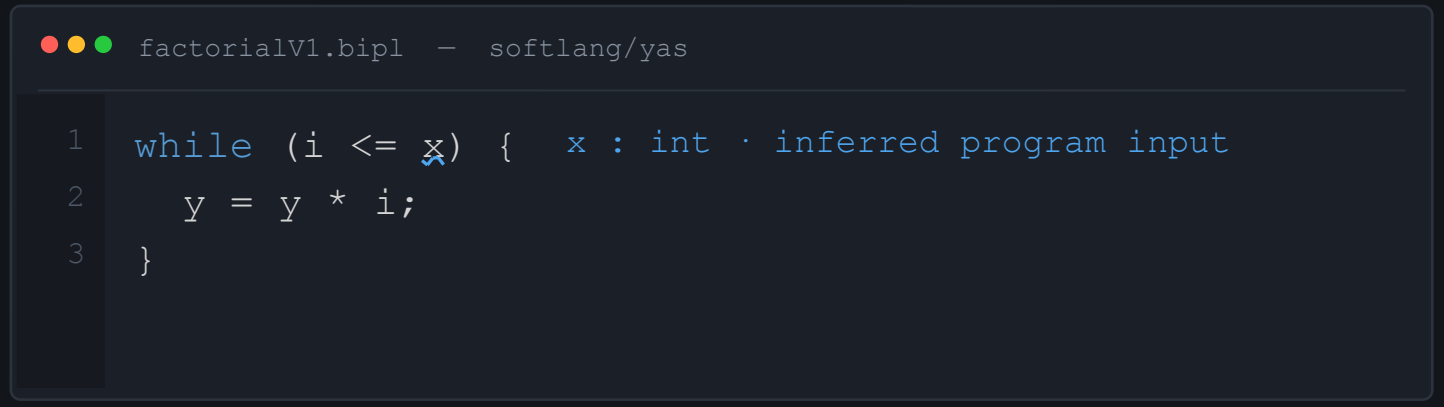
bipl-lsp

Editing BIPL, properly.

A language server that makes a tiny language feel like a real one.



The claim: even five statements hide two real language problems.

Vivekananda Reddy Godala SLE Assignment 3 · Option LSP.2 · Prof. Ralf Lämmel · 2026-07-13

The language: BIPL

“A trivial imperative programming language … a small fragment of C.” — softlang/yas

division.bipl

```
// Euclidean division
{
  q = 0;
  r = x;
  while (r >= y) {
    r = r - y;
    q = q + 1;
  }
}
```

Three facts make it interesting to tool

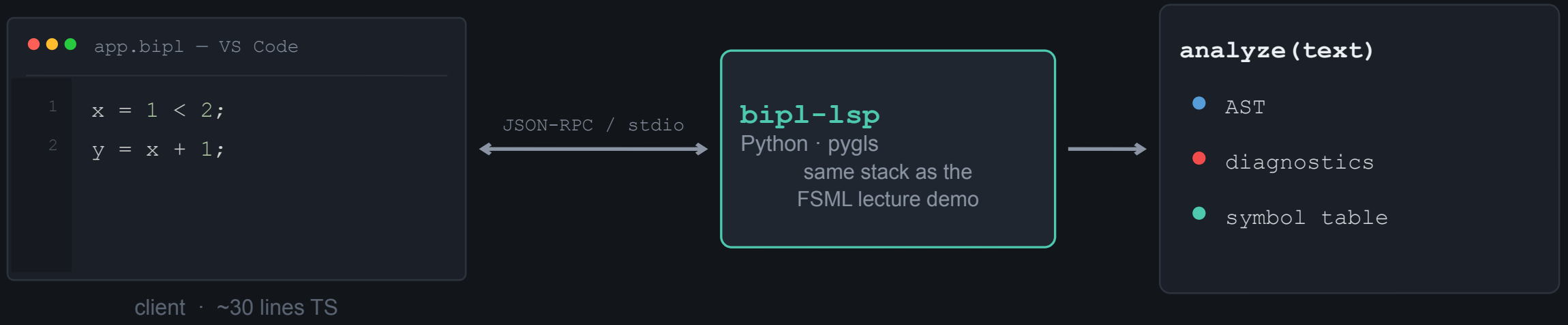
No declarations. You never write int x. The editor must infer every type — nobody states it.

No boolean literals. No true / false keyword. Booleans exist only as results of comparisons and && || !.

Inputs live in the store. No input statement — a program just reads variables already in memory. That is why samples read x without assigning it.

I didn't invent an architecture — I instantiated the lecture's

“Editors own the UI; servers own the language knowledge.” — LSP lecture (softlang/yas, FSML)



One pure function does the thinking. Hover, definition, rename, completion, and the outline are all just cheap queries on its result — cached per document version, exactly as the lecture's sequence diagram shows.

Where the squiggles come from

1 — Type checking

A variable's type comes from its first assignment; a constrained use pins an input's type.

types.bipl

```
1  x = 5;
2  x = 1 < 2;   int can't become bool
```

infer.bipl

```
1  while (i <= x)   x : int (inferred)
```

2 — Definite assignment

A read is safe only if assigned on every path. An if needs both branches; a while body may run zero times.

flow.bipl

```
1  if (c) y = 1;
2  z = y;   y maybe uninitialized
3
4  while (c) v = 1;
5  u = v;   v maybe uninitialized
```

In the book's terms (Ch. 9): our checker implements the BIPL judgments $m \vdash e : T$ and $m \vdash s$ over a context of variable-type pairs — and completes the operator set the YAS Haskell checker left as "TODO".

Not every unassigned read is a bug

BIPL reads its inputs from the store. So a naive “read-before-write = error” rule would flag `factorialV1.bipl` — an official sample — as broken. **The language would reject its own textbook.**

PROBLEMS

✖ Error	`type-error`	conditions must be boolean; a variable can't change type
⚠ Warning	`maybe-uninitialized`	assigned somewhere, but not on every path to this read
ⓘ Information	`input-variable`	never assigned anywhere → a program input; this is fine

12/12

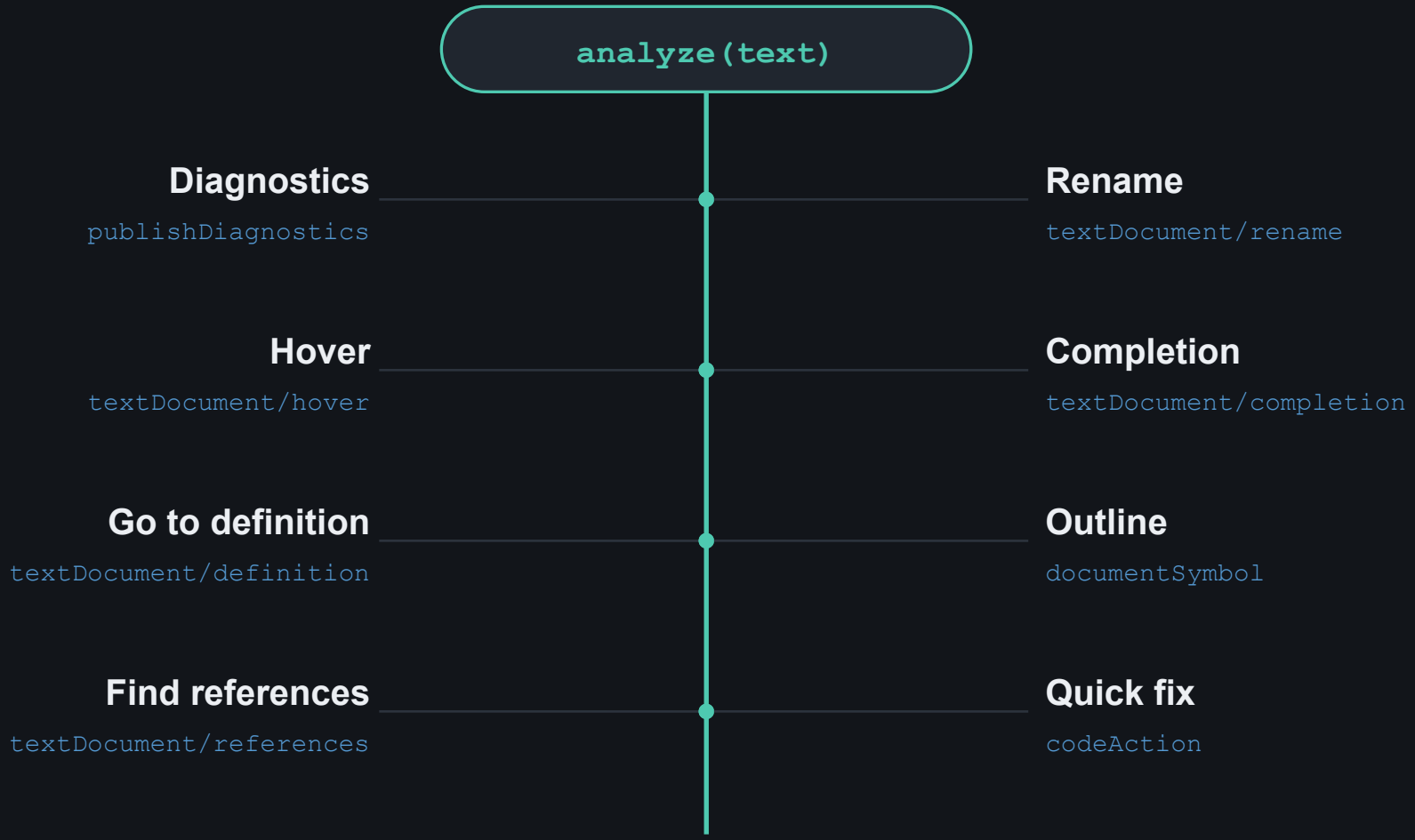
official YAS samples
analyse with zero errors

— and a test locks it in,
so it can never regress.

Respecting the language's own semantics — instead of imposing a naive rule — is what makes the server “interesting.”

One analysis, every feature

No feature has its own parser. Each is a query on the same analyze() result.



Demo

bad.bipl

```
1  {
2    x = 5;
3    if (x) y = 1;   condition must be boolean
4    z = y + 1;   y maybe uninitialized
5    w = (2 < 3) + 4;   '+' expects integers
6  }
```

PROBLEMS ✖ 3 errors ⚠ 2 warnings

1. **good.bipl**
   blue input hints; hover shows types inferred from use
2. **bad.bipl**
   3 type errors + 2 data-flow warnings, live
3. **Navigate**
   hover · go-to-definition · find references
4. **Refactor**
   F2 rename (rejects invalid names) · quick fix
5. **Live typing**
   break factorial, watch diagnostics move as you type

Three things to remember

1 **LSP decouples smartness from the editor.**
One Python analysis serves any LSP editor; the VS Code client is 30 lines.

2 **A small language is not trivial tooling.**
Five statements still forced type inference without declarations and branch-sensitive data flow.

3 **The language's own samples were the spec.**
They told me inputs aren't errors — and now they guard that decision as tests.

---

code · tests · demo files `github.com/VivekanandaReddy3/sle-bipl-lsp`